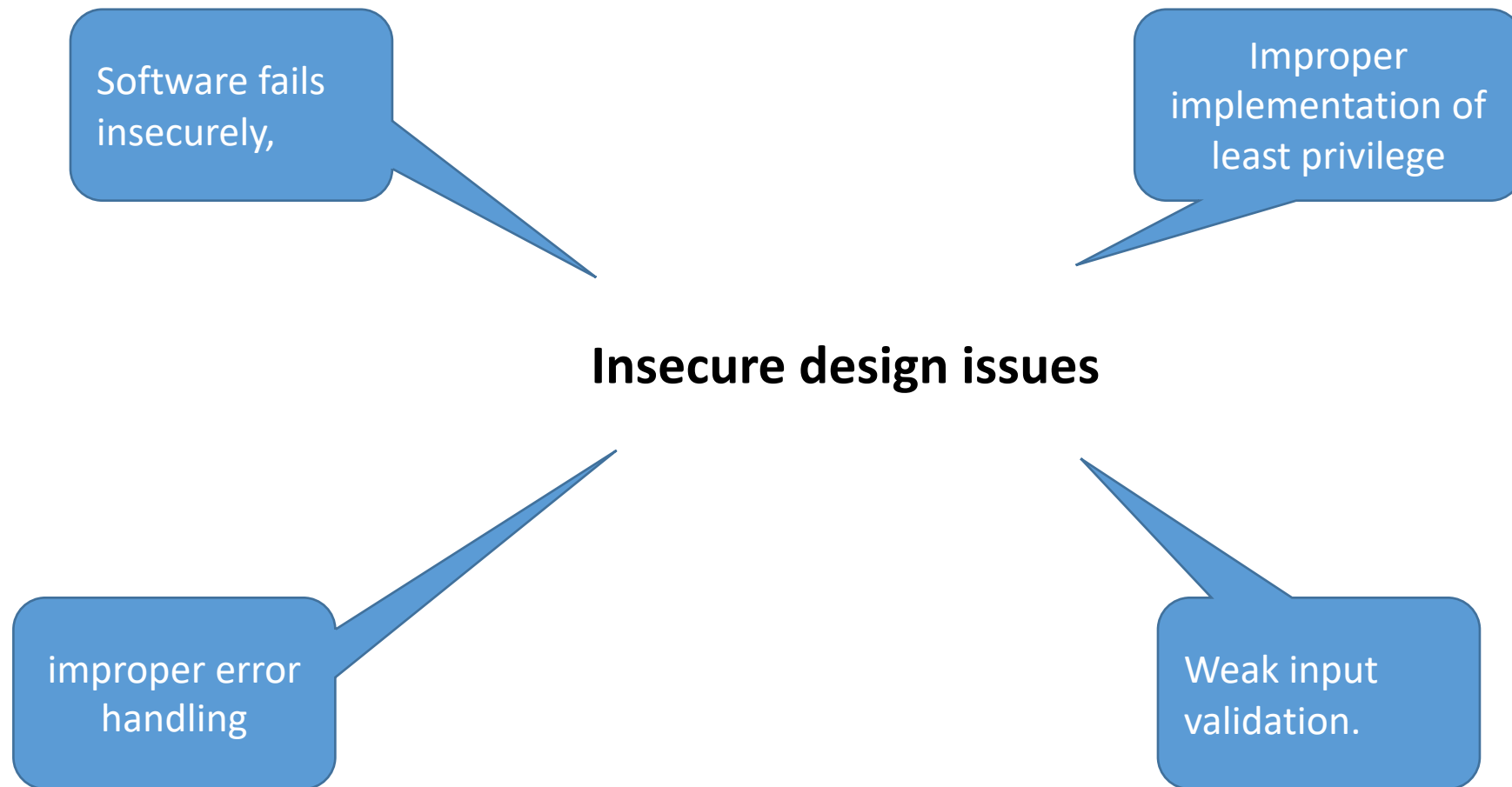


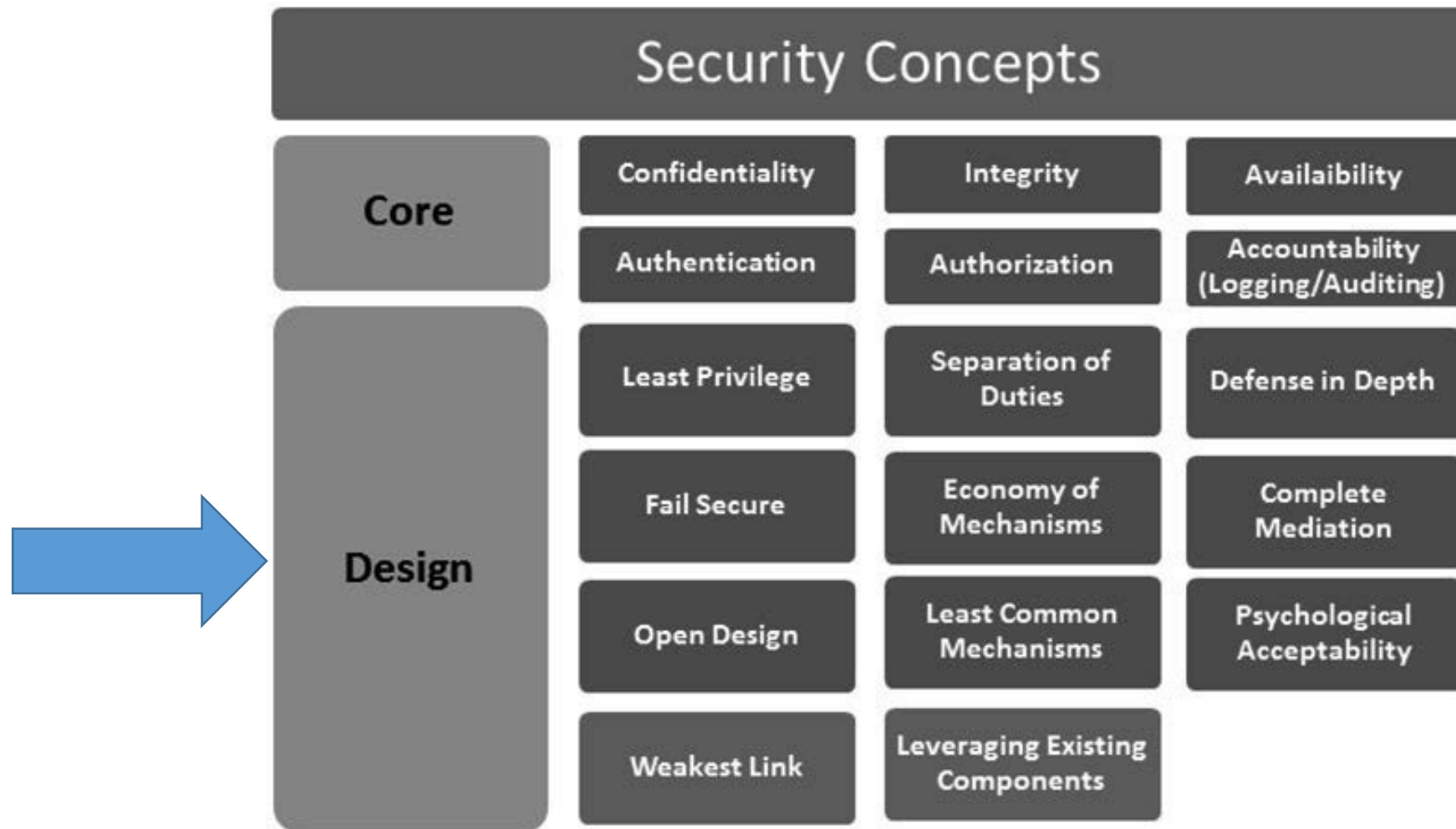
Secure Software Design and Development (CY-321)

Secure Design Processes

Secure Design Requirements

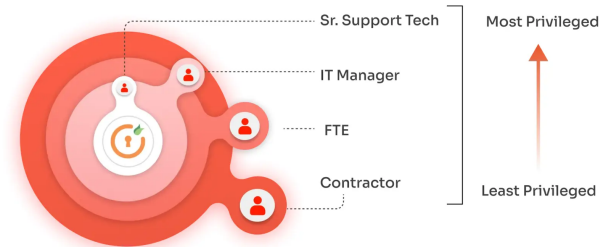
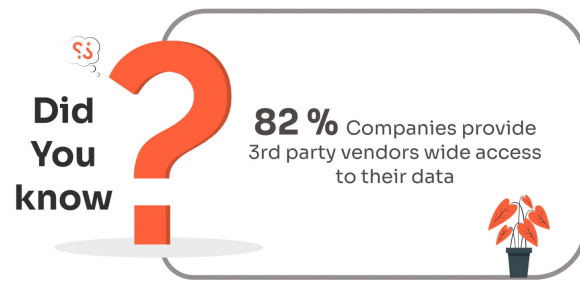


Secure Design Requirements



Secure Design Requirements

Least Privilege



The Principle of Least Privilege

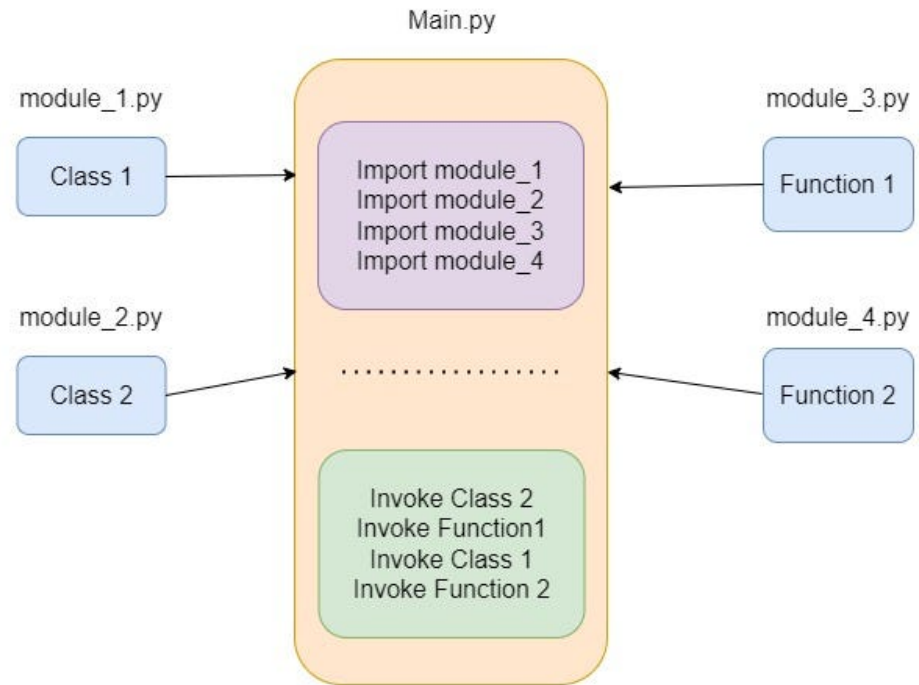
Need-to-know

Modular programming

Secure Design Requirements

Least Privilege

Modular programming



High Cohesion

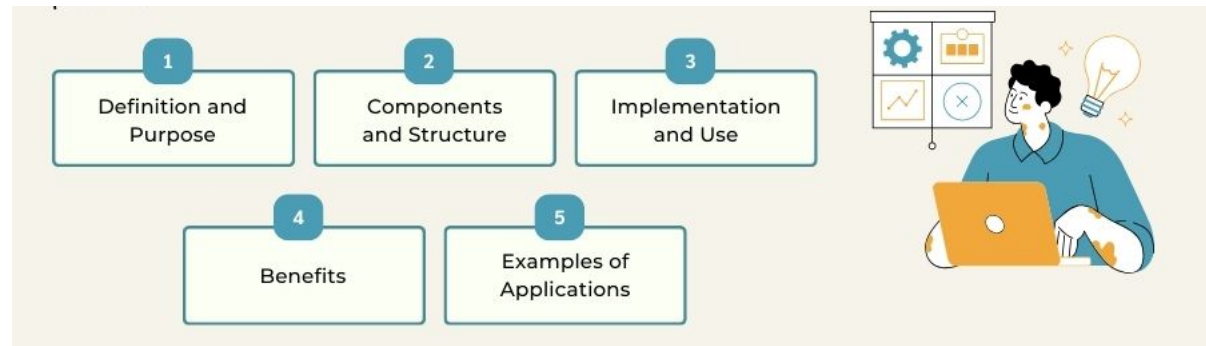
Loose Coupling

Secure Design Requirements

Separation of Duties

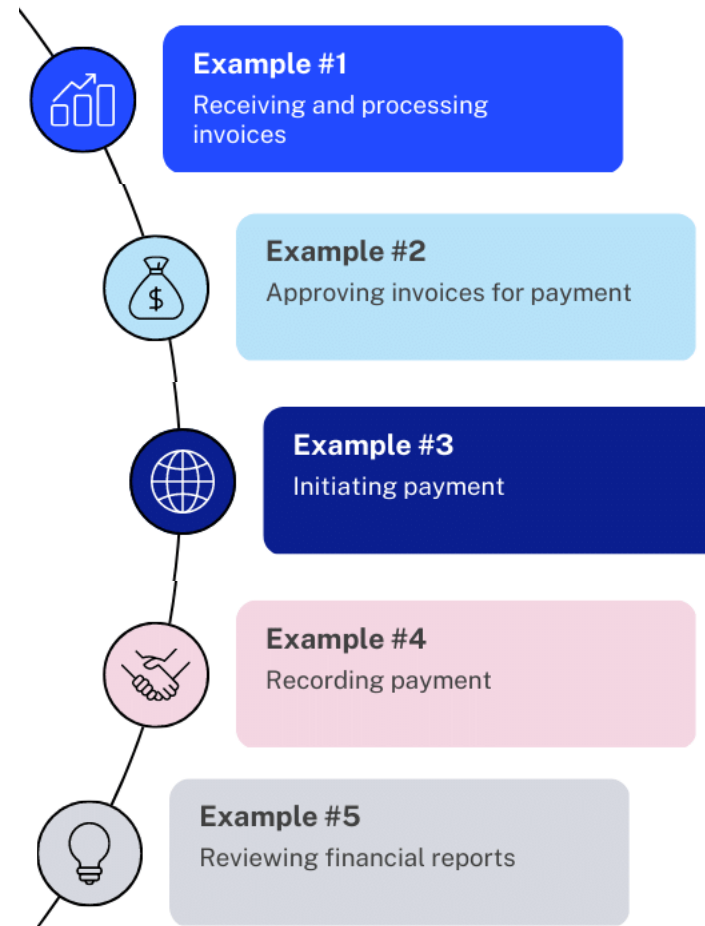
Software functionality into two or more conditions, all of which need to be satisfied before an operation can be completed

Software **Deployment, Operations, Maintenance, and Disposal** chapter.



Secure Design Requirements

Separation of Duties



Secure Design Requirements

Defense in Depth



Not putting all the eggs in one basket

Use of Input Validation & Prepared Statements
to Prevent Injection Attacks

Defending Against Cross-Site Scripting (XSS)
with Output Encoding & Validation

Security Zones for Access Control

Secure Design Requirements

Fail Secure



when attacked and is rapidly *recoverable* into a normal business

The user is denied access by default and the account is locked out after the maximum number (clipping level) of access attempts is tried

Errors and exceptions are explicitly handled and the error messages are non-verbose in nature

Not designing the software to ignore the error and resume next operation

Secure by Design,

Secure by Default

Secure by Deployment

Secure Design Requirements

Economy of Mechanisms



“bells-and-whistles”

Unnecessary functionality or unneeded security mechanisms should be avoided.

Strive for simplicity

Strive for operational ease of use

Secure Design Requirements

Complete Mediation

Not checking access rights
each time a subject requests
access to objects violates the
principle of complete
mediation

Complete mediation also
augments the protection
against the weakest link

Credit Card's Billing Name & Address:

First Name:

Last Name:

Address:

City:

State/Province:

Zip/Postal Code:

Country:

Process Now

(do not click more than once)

Secure Design Requirements

Open Design

Kerckhoff's principle

All information about the crypto system is public knowledge except the key, and the security of the crypto system against cryptanalysis attacks is dependent on the secrecy of the key



Secure Design Requirements

Open Design



The Opposite??

The security of your software should not be dependent on the *secrecy of the design*

Security through obscurity should be avoided

The design of protection mechanisms should be open for scrutiny by members of the community

Secure Design Requirements

Psychological Acceptability



When user feel that security is usually very complex

Name:

Phone Number:

**Invalid Field**
Please enter a phone number in the format:
(###) ###-####

- are easy to use,
- do not affect accessibility, and
- are transparent to the user

Secure Design Requirements

Weakest Link



"A chain is only as strong as its weakest links."



"A chain is only as weak as its strongest links."

"Single Point of Failure"

Software must be architected so that there is no single source of complete compromise

Secure Design Requirements

Leveraging Existing Components

Service Oriented Architecture (SOA) is prevalent in today's computing environment

Avoid custom implementations of cryptographic functionality are also determined often to be the weakest link.

the attack surface is not increased, and no newer vulnerabilities are introduced

Secure Design Requirements

Interface Design

Application Programming Interfaces (API)



Secure Design Requirements

Interface Design

Out-of-Band Interface

Log Interfaces

Security Management Interfaces (SMI)

Balancing Secure Design Principles

"SSO can heighten user experience and increase psychological acceptability, it contradicts the principle of complete mediation and so a business decision is necessary to determine the extent to which SSO is designed into the software or to determine that it is not even an option to consider "

Design Processes

When you are designing software with security in mind, certain security processes need to be established and completed

Attack surface evaluation

The measure of its exposure of being exploited by a threat agent

Design Processes

When you are designing software with security in mind, certain security processes need to be established and completed

These processes are to be conducted during the initial stages of the software development project

Attack surface evaluation

Threat modeling

documentation

Attack Surface Evaluation

A software or application's attack surface is the measure of its exposure of being exploited by a threat agent

Weaknesses in its entry and exit points that a malicious attacker can exploit to his or her advantage



Attack Surface Evaluation

It is particularly important to threat model **legacy software**

Why?

Targets and Enablers are resources that an attacker can leverage to construct an attack against the software.

The two kinds of targets and enablers are **processes and data**. Browsers, mailers, and servers are examples of **process targets and enablers**, while files, directories, and registries are examples of **data targets and enablers**

Channels and Protocols are mechanisms that allow for communication between two parties

Message-passing channels and Shared-memory channels



Design Processes

Relative Attack Surface

Microsoft Cyber Security Program Manager
Michael Howard

Measuring Relative Attack Surfaces

Michael Howard, Jon Pincus, and Jeannette M. Wing

Abstract—We propose a metric for determining whether one version of a system is more secure than another with respect to a fixed set of dimensions. Rather than count bugs at the code level or count vulnerability reports at the system level, we count a system’s *attack opportunities*. We use this count as an indication of the system’s “attackability,” likelihood that it will be successfully attacked. We describe a system’s *attack surface* along three abstract dimensions: targets and enablers, channels and protocols, and access rights. Intuitively, the more exposed the system’s surface, the more attack opportunities, and hence the more likely it will be a target of attack. Thus, one way to improve system security is to reduce its attack surface.

To validate our ideas, we recast Microsoft Security Bulletin MS07-005 using our terminology and we show how

ferent versions of the same system.

A. Motivation

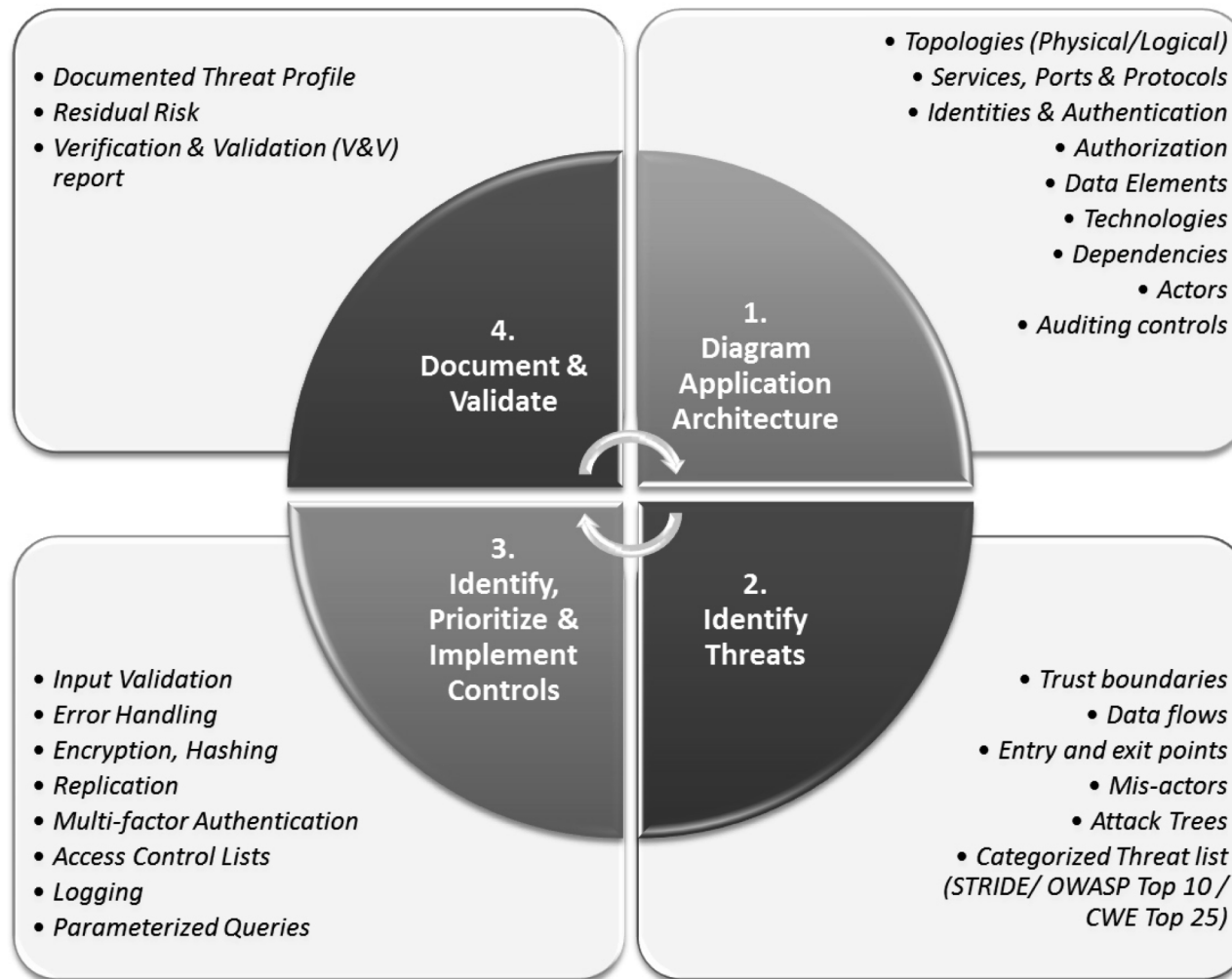
Our work is motivated by the practical problem faced in industry today. Industry has responded to demands for improvement in software and systems security by increasing effort¹ into creating “more secure” products and services. How can industry determine if this effort is paying off and how can we as consumers determine if industry’s effort has made a difference?

Our approach to measuring relative security between systems is inspired by Howard’s informal notion of *relative attack surface* [1]. Howard identified 17 “attack vec-

Not Isolation, But Comparision

“More doors in a house mean more places to lock.”

Threat Modeling Process



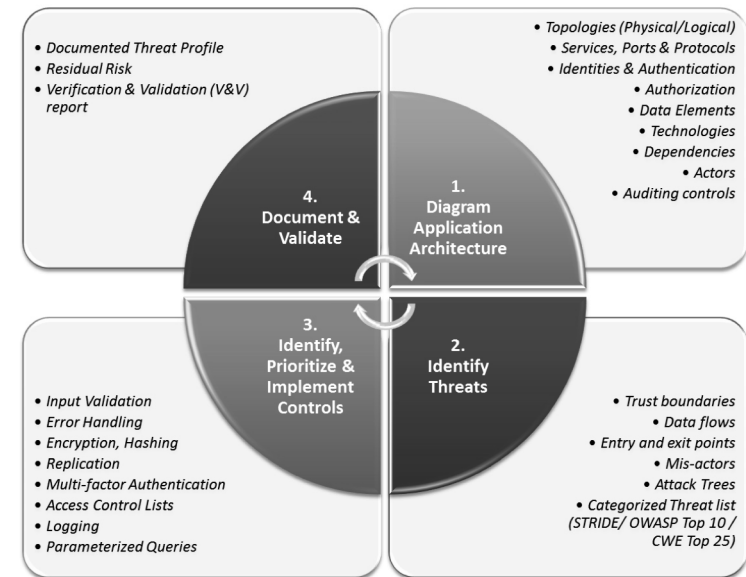
Threat Modeling Process

Model Application Architecture

Identify the software system, its components, and its interactions

Create Data Flow Diagrams (DFDs) or Architecture Diagrams to visualize data movement and trust boundaries.

Define assets that need protection (e.g., sensitive data, credentials, APIs).



Threat Modeling Process

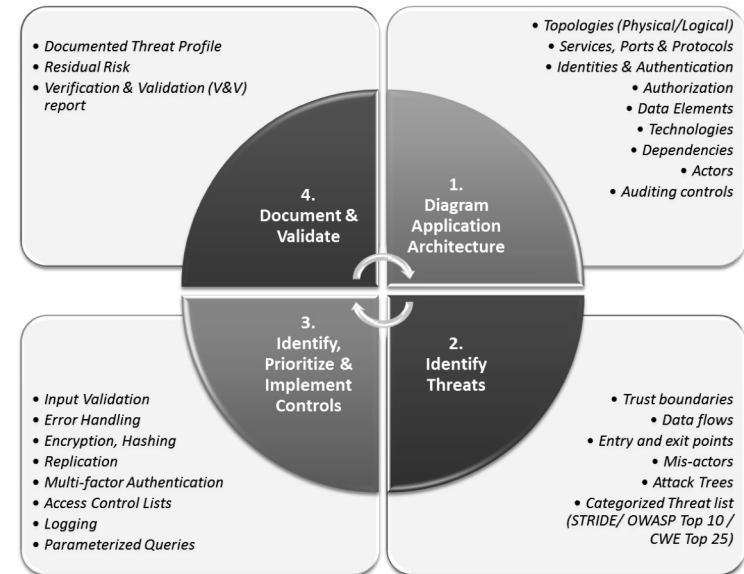
Model Application Architecture

Identify the Physical Topology

Gives insight into where and how the applications will be deployed

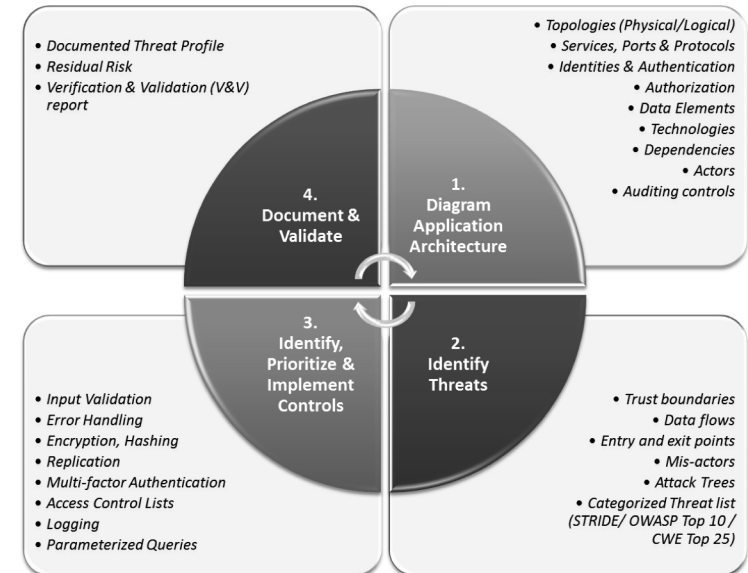
Identify the Logical Topology

Examples include forms based, certificate based, token based, biometrics, single sign-on, multi-factor,



Threat Modeling Process

Model Application Architecture



Identify Human and Non-Human Actors of the System

Examples include customers, sales agents, system administrators, database administrators, etc.

Identify Data Elements

Examples include product information, customer information, etc.

Threat Modeling Process

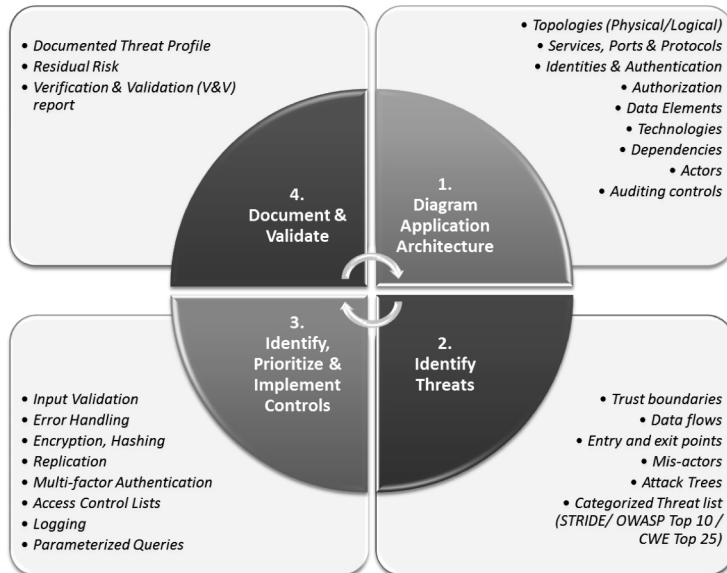
Model Application Architecture

Generate a Data Access Control Matrix.

The rights and privileges that the actors will have on the identified data elements

Identify the technologies that will be used in building the application.

Identify external dependencies.



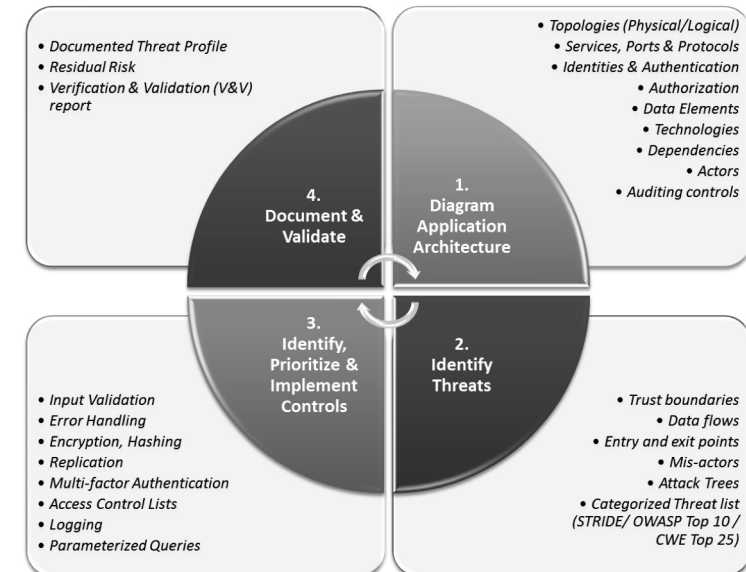
Threat Modeling Process

Identify Threats

This step involves **thinking like an attacker** and listing potential security threats based on the system design.

Identify Trust Boundaries

Identify actions or behavior of the software that is allowed or not allowed

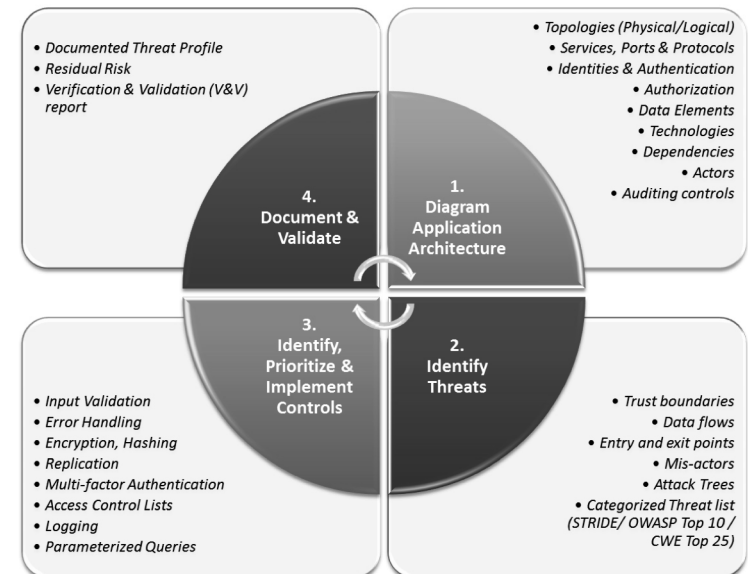


Threat Modeling Process

Identify Threats

Identify Entry Points

Each entry point can be a potential threat source

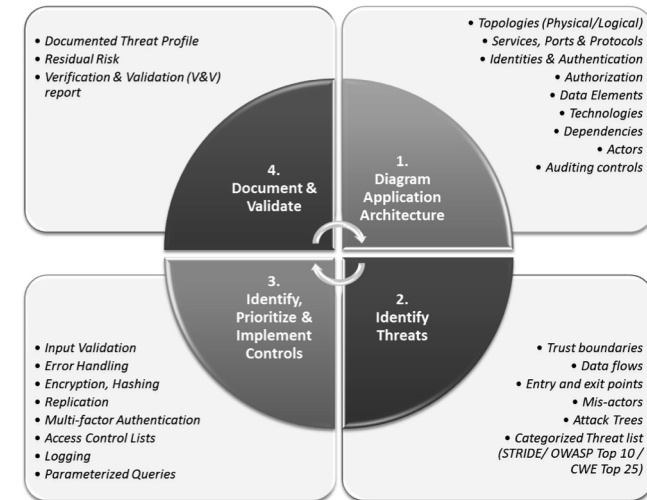


Threat Modeling Process

Identify Threats

Identify Privileged Functionality

All administrator functions and critical business transactions are identified.



Identify Data Flows

Data flow diagrams (DFDs) and sequence diagrams assist in the understanding of how the application will accept, process, and handle data

Identify Exit Points

Exit points can be the source of information leakage and need to be equally protected

Threat Modeling Process

Document and Validate

The importance of documenting the threat model cannot be underestimated

Why?

because threat modeling is iterative and,
through the life cycle of the project

Threat Modeling Process

STRIDE

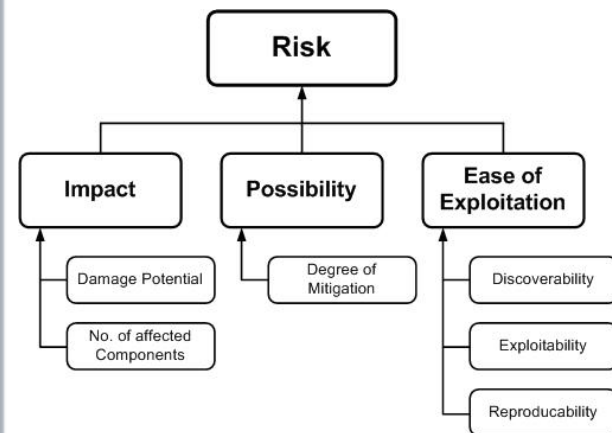
A threat modeling methodology that is performed in the design phase of software development which threats are grouped and categorized



Threat Modeling Process

DREAD

Rating methodology that is often used in conjunction with STRIDE



Damage potential What will be the impact upon exploitability?

Reproducibility What is the ease of recreating the attack/exploit?

Exploitability What minimum skill level is necessary to launch the attack/exploit?

Affected users How many users will be potentially impacted upon a successful attack/exploit?

Discoverability What is the ease of finding the vulnerability that yields the threat?

DREAD is still commonly taught, but it has **fallen out of favor in practice**, and Microsoft itself moved away from it. Scores are subjective

Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby